# pqscan: A Fast, Chainable Cipher-Configuration Scanner for TLS, QUIC, and Post-Quantum Readiness

**Shaineel Singh (shain.singh@owasp.org)**

pqscan project · Melbourne, Australia

*Revision 1.0 – 5 November 2026 · Work in progress. Full product specification companion: "pqscan – Product Requirements Document" (v0.3, 2026-10-15).*

**Abstract.** The NIST post-quantum standardisation wave moved post-quantum readiness out of research papers and into the live cipher-configuration layer of TLS and QUIC: hybrid key-exchange groups are registered, browsers negotiate `X25519MLKEM768` by default, and ML-DSA and composite certificates are entering the registries and the certification authorities. The transition runs in a layer that security teams audit least and that the tooling covers worst: exhaustive probes are interactive and one target at a time, fast large-scale scanners carry thin TLS insight and no post-quantum classification, and the accessible tools are serial, not pipe-first, and pre-date the hybrid groups. We present pqscan, a single static Go binary that reads the complete cipher-configuration plane of TLS over TCP and TLS 1.3 over QUIC – protocol versions, cipher suites, key-exchange groups, signature algorithms, certificate chains and ALPN – in four operating modes (fast, one engineered handshake per target; deep, the offer-one/observe-accept matrix; pq, forced post-quantum group probing; and selftest), and emits chainable output: stdin to stdout, schema-versioned JSON lines, a fixed 25-column CSV, and deterministic exit codes pinned by golden tests. Classification tables are generated from the IANA registry rather than hand-edited; every negotiated parameter is scored `pq`, `pq_hybrid`, `classic`, or `obsolete`; per-host readiness combines negotiated hybrid key exchange, observable capability, modern hygiene, and certificate signals into a rated score with a harvest-now-decrypt-later flag. We describe the language decision and its kill-switch criteria, the bounded five-stage concurrency architecture with its throughput model, budgets, and adaptive-concurrency controls, the verification strategy carried as CI, and the conservative rate defaults that enforce scanning ethics by construction rather than by policy claim.

*Keywords* – post-quantum cryptography · TLS 1.3 · QUIC · cipher configuration · readiness ratings · harvest-now-decrypt-later · chainable command-line tools · protocol census · Go · pipeline architecture.

# Table of Contents


- 1. Introduction
  - 1.1 The problem: the transition runs in the cipher-configuration layer
  - 1.2 The design thesis in one sentence
  - 1.3 Why now
  - 1.4 Contributions
  - 1.5 Paper structure
- 2. Background and Related Work
  - 2.1 The field, surveyed in full
  - 2.2 What we borrow, with credit
  - 2.3 The synthesis rule: compose, never rebuild
- 3. Design Goals and the Language Decision
  - 3.1 Goals
  - 3.2 Non-goals
  - 3.3 Go or Rust: a decision matrix, not a taste
- 4. The Tool: Targeting, Modes, and the Probe Set
  - 4.1 Targeting and discovery
  - 4.2 The four modes
  - 4.3 Protocol coverage
  - 4.4 The handshake-engine capability checklist
  - 4.5 Certificate analytics
  - 4.6 Post-quantum classification and the readiness rubric
- 5. The Output Contract and Chainability
  - 5.1 The dialect rules
  - 5.2 The record
  - 5.3 The CSV projection and the output flags
  - 5.4 The chaining reference implementations
  - 5.5 The CLI surface
- 6. Concurrency Architecture
  - 6.1 Where time goes: the throughput model
  - 6.2 The pipeline shape: five bounded stages
  - 6.3 Budgets and fairness
  - 6.4 Deep-mode scheduling: the probe plan
  - 6.5 The QUIC engine
  - 6.6 No head-of-line anywhere
  - 6.7 Memory, allocation, and GC discipline
  - 6.8 Benchmarks and SLOs
- 7. Test and Verification Strategy
  - 7.1 Test-first is the merge gate
  - 7.2 Correctness fixtures
  - 7.3 Protocol truth
  - 7.4 Performance and resilience
  - 7.5 Rate safety by construction
- 8. Success Metrics
- 9. Risks, Limitations, and Future Work
  - 9.1 The risk register
  - 9.2 Stated limitations
  - 9.3 Future work
- 10. Ethics, Authorship, and Use
- 11. Conclusion

• 12. References

# 1. Introduction

## 1.1 The problem: the transition runs in the cipher-configuration layer

The 2024–2026 NIST post-quantum standardisation wave — FIPS 203 (ML-KEM) [1], FIPS 204 (ML-DSA) [2], FIPS 205 (SLH-DSA) [3], with HQC joining as a selected key-encapsulation mechanism in 2025 [4] — moved post-quantum readiness from the research papers into live TLS deployments. Browsers negotiate the `X25519MLKEM768` hybrid group by default, and Go's standard TLS library enabled it by default in version 1.24 [6], [18]. RFC 10024 registers the ECDHE-MLKEM hybrid groups [5]; ML-DSA and the composite ML-DSA schemes, with post-quantum certificates, are entering the registries and the certification authorities [6]. The transition runs in the cipher-configuration layer — the group a server accepts, the algorithm it signs with, the certificate it offers. Security teams audit that layer least, and the tooling covers it worst.

The current tools take three shapes, and none fills the gap. The **exhaustive** shape (gnutls-cli-debug, testssl.sh, TLS-Scanner [7], [8], [9]) carries the deepest probe set but is interactive, one target at a time, slow, and weak or awkward on post-quantum groups. The **fast/scalable** shape (zgrab2, masscan followed by deeper tools [10], [11], [12]) carries the scale but offers thin per-target TLS insight and no post-quantum classification. The **accessible** shape (cryptonice, sslscan [13], [14]) offers friendly output but is Python/Perl-based, serial, not chainable, and pre-PQC.

No modern, fast, small, dependency-free tool carries the whole set: exhaustive cipher-configuration reads of the gnutls-cli-debug class; post-quantum readiness rates on a stated rubric; natural pipes into the subfinder → httpx → nuclei ecosystem [15], [16], [17]; and scale from one domain to a /16 with no change of model. `pqscan` is that tool: a small binary that reads TLS and QUIC cipher configuration and post-quantum readiness on every target, stdin → stdout, one JSON record per line, chainable. It is comfortable on a single domain, it completes a /24 without drift, it is viable for one host scanning a whole IPv4 /8, and it treats the whole-internet fast pass as a documented multi-box recipe with shards rather than a bundled one-liner.

## 1.2 The design thesis in one sentence

**A single static binary reads the complete cipher-configuration plane of TLS and QUIC — every version, suite, group, and signature algorithm a server accepts, plus post-quantum readiness on a stated rubric — at speeds compatible with the discovery toolset, and composes into that toolset instead of replacing it.**

Two commitments follow. First, the scale is a product requirement, not an implementation note: the pipeline architecture (Section 6) precedes the probes, because a cipher scanner is not a web scraper — its work is thousands of short, RTT-dominated, failure-rich round-trips whose failures carry information. Second, output is a contract, not a convenience: the JSONL record is schema-versioned, the CSV projection is column-pinned, and both are pinned by golden tests so that pipelines built on the tool keep running across releases (Section 5).

## 1.3 Why now

Three curves intersect in 2026:

1. **The standards landed and the registries moved.** FIPS 203, 204, and 205 are published [1], [2], [3]; the hybrid groups are registered as RFC 10024 [5]; the ML-DSA certificate OIDs are specified as RFC 9881 (Section 4.5); the composite and SLH-DSA signature schemes sit in the TLS signature-scheme registry [6], [33]. A scanner that cannot speak the new codepoints can no longer state a host's readiness at all.
2. **The default clients negotiate post-quantum today.** Browsers ship `X25519MLKEM768` by default, and crypto/tls has done the same since Go 1.24, with the negotiated hybrid group directly observable in

`ConnectionState.CurveID` from Go 1.25 [18]. The negotiated plane is measurable with no C bindings and no custom stacks.

3. **Harvest-now-decrypt-later sets the clock.** A server whose key exchange is purely classical today produces recorded traffic that stays decryptable to a future cryptographically relevant quantum computer. Readiness is not a curiosity about a future handshake; it is a fact about traffic already leaving the building – the reasoning behind the tool's `hndl_risk` flag (Section 4.6).

## 1.4 Contributions

1. **A complete server-side cipher-configuration read, rethought for automation (Section 4):** the gnutls-cli-debug probe set carried by a non-interactive client – version-by-version and suite-by-suite "offer-one / observe-accept" enumeration, key-exchange-group enumeration including uTLS-constructed groups stock stacks refuse to build, signature-algorithm negotiation, and the feature checks (HRR, 0-RTT, resumption, fallback SCSV, renegotiation) that the exhaustive class knows and the scalable class lacks [7], [24].
2. **Post-quantum classification as a first-class output (Section 4.6):** a four-class taxonomy over key exchange and over signatures and certificates, classification tables generated from the IANA registry rather than hand-edited, a fixed printable readiness rubric with banded ratings, and the harvest-now-decrypt-later flag [33].
3. **Full-protocol coverage for TLS over TCP and TLS 1.3 over QUIC (Sections 4.3, 6.5):** including post-quantum group negotiation observed inside QUIC, the RFC 9369 transport-version rung, and a functional HTTP/3 rung that separates "ALPN h3" from "h3 serves requests" [28], [29], [23].
4. **The chainability contract (Section 5):** stdin→stdout all the way with one grammar in and schema-versioned JSONL, column-pinned CSV, and text out; import adapters for httpx, naabu, nuclei, and masscan lines [15], [16], [19], [11]; reference pipe recipes that run verbatim.
5. **A concurrency architecture designed for the failure case and the tail (Section 6):** a five-stage bounded pipeline on a measured throughput model ( `T ≈ W/L` ), explicit budgets, AIMD adaptive concurrency with per-subnet and per-ASN circuit breakers, and a no-head-of-line discipline – the design that makes a 50,000-connection scanner safe rather than merely fast.
6. **A verification strategy carried as CI (Section 7):** a test-first merge gate [39]; a badssl.com fixture matrix [40]; differential evaluation against gnutls-cli-debug and sslyze [7], [20]; protocol-truth cross-checks (Wireshark-visible QUIC handshakes, raw-ClientHello parsing); and a performance suite that enforces the SLO table per release cut.
7. **A stated-position argument (Sections 2, 9):** compose, never rebuild – pqscan occupies exactly one altitude, the cipher-configuration plane, and the tool leaves discovery, HTTP content work, and templating to the tools that own those altitudes better.

## 1.5 Paper structure

Section 2 grounds the design in the field it sits in – the three tool shapes, what is borrowed, and the synthesis rule. Section 3 records the goals, the non-goals, and the language decision with its kill-switch criteria. Section 4 presents the tool surface: targeting, the four modes, the protocol coverage, the handshake-engine capabilities, certificate analytics, and the post-quantum classification rubric. Section 5 the output contract and the chaining reference implementations. Section 6 the concurrency architecture, ending in the CI-enforced SLO table. Section 7 the verification strategy. Section 8 the success metrics, Section 9 the risks, the stated limitations, and the future work, Section 10 ethics, authorship, and use, and Section 11 concludes.

# 2. Background and Related Work

## 2.1 The field, surveyed in full

Table 1 states, per tool, what each tool does well and why pqscan does not adopt it; the rule of engagement is the spec's own: borrow the capability, state the reason, and compose rather than rebuild.

**Table 1:** The TLS tooling field, surveyed in full (capability breadth and non-adoption reasons)

| Tool | What it does well | Why pqscan does not adopt it |
|---|---|---|
| cryptonice (F5 Labs) | Broad scan menu (TLS, HTTP, HTTP/2, DNS, JARM), readable JSON report, sslyze-backed rigour | Python + async per-target model; not built for a 10k/s fan-out; PQ groups and certs absent; not pipe-first |
| gnutls-cli-debug | The reference-standard exhaustive per-protocol/per-suite probing of a single server | One target at a time; interactive; no scripting of the probe matrix; no PQ groups |
| testssl.sh | Very complete suites, legacy protocols, and vulnerability checks; portable bash | Shell; slow per target; PQ groups treated as opaque hex; output not pipeline-native |
| sslyze | Solid Python TLS probing library, per-version suite enumeration | Python, serial; PQ gaps the same as cryptonice |
| zgrab2 | The scale leader: Go, modular scanners, banner grab, JSON, pipes into masscan/zmap | TLS probe depth is shallow by design (one or few handshakes); no PQ classification; probe matrix not user-extensible |
| masscan / zmap | SYN-port discovery at extreme rates | Not a TLS client; out of scope – they feed pqscan (the Section 5.4 recipe) |
| TLS-Scanner (TUD) | Research-grade exhaustive protocol analysis | Academic codebase; no packaging or usability for operations |
| httpx / nuclei / naabu | The chainability and output dialect, resume, stats, rate flags | Not TLS-deep (httpx performs one handshake); nuclei's TLS data is template-shaped; naabu is ports only |
| Censys / Shodan | Mass passive TLS datasets including PQ group observations | Remote-only datasets; no active per-suite probing; no on-demand targeting |

Two of the rows deserve emphasis. **zgrab2 leads on raw churn, and remains the model for the streaming-output shape** – the shard, resume, and stats semantics pqscan adopts are zgrab2's [10]. **The exhaustive class holds the depth that makes a ratings rubric defensible:** gnutls-cli-debug's offer-one/observe-accept method is the correctness standard for per-suite truth, and pqscan re-implements that method as a scripted, parallelised, non-interactive plan (Section 4.2, Section 6.4) [7].

## 2.2 What we borrow, with credit

The design is a composition with acknowledged ancestry. From gnutls-cli-debug, the exhaustive probe semantics [7]; from zgrab2, the shard/resume/stats operating posture and the streaming JSONL record [10]; from httpx and nuclei, the output dialect – single-line human output, `-jsonl`, deterministic exit codes, `-stats` – that makes a scanner pipe-native [15], [16]; from masscan and zmap, the discovery layer pqscan deliberately does not rebuild [11], [12]; from cryptonice, the readable per-host report shape and the resumption-rate check (Section 4.4) [13]; from testssl.sh, the fixture-minded completeness expectation (Section 7.2) [8]. The passive vantage is complementary, not competing: the Censys and Shodan datasets cover breadth at dataset scale [21], [22], while the active plane answers per-target, on-demand truth. Where a project owns an altitude better than pqscan ever

will – port discovery at the extreme scale, host discovery, HTTP content work, template engines – the design defers to it explicitly.

### 2.3 The synthesis rule: compose, never rebuild

The synthesis rule sits behind the survey and fixes the tool's position: **port discovery stays with naabu/masscan/zmap (compose, never rebuild); host discovery stays with subfinder; HTTP content work stays with httpx; nuclei keeps its template engine.** pqscan covers exactly one altitude: the *cipher-configuration plane* – TLS 1.x and QUIC handshake parameters, certificates, and PQ classification – at speeds compatible with those tools. The practical consequence is the pipeline shape of Section 5.4: a whole engagement's TLS/PQC stage reads as two pipes, not a new platform.

## 3. Design Goals and the Language Decision

### 3.1 Goals

Seven goals define v1, stated in the companion specification as G1–G7 [6]:

- **G1 – Full TLS cipher-config enumeration (server side).** All protocol versions offered/probed (SSLv3 optional; TLS 1.0–1.3 core), all cipher suites testable suite-by-suite, key-exchange groups, signature algorithms, certificate-chain analytics – the gnutls-cli-debug probe set, rethought for automation.
- **G2 – PQC classification as a first-class output.** Every negotiated or observed parameter is scored `pq | pq_hybrid | classic | obsolete`; a per-host readiness rating with reasons; and a harvest-now-decrypt-later risk flag.
- **G3 – Full-protocol coverage: TCP TLS + QUIC.** TLS over TCP for HTTP/1.1 and h2 (ALPN-driven), and TLS 1.3 over QUIC (h3) on UDP/443, including PQ group negotiation inside QUIC.
- **G4 – Chainable like the import toolset.** Plain hosts, `ip:port`, and URLs in on stdin; `-json/-jsonl/-csv` out on stdout; deterministic exit codes; pipe-in/pipe-out all the way.
- **G5 – Fast past any TLS tool today.** A multi-gigahertz handshake engine: at least 2,500 handshakes/s per core in fast mode; 100k+ in-flight connections; p95 per-target completion under 15 s in fast mode under load.
- **G6 – Small.** Single static binary of 15 MB or less, no configuration required; `pqscan -u example.com` is a complete invocation.
- **G7 – Reverse-DNS-aware discovery.** Given IP ranges, enrich with PTR records and pick SNI candidates automatically, with several deduplicated SNI strategies.

### 3.2 Non-goals

The v1 boundary is doctrine, not omission:

- Not a vulnerability scanner with exploit logic. The heartbleed/ROBOT/CCS-injection classes ship as a separate opt-in legacy module, reusing classic public probe logic – they are not the reason the tool exists.
- No fuzzing, and no downgrade/CVE classification reports in the style of security-metric dashboards. Ratings are configuration facts, not CVE claims.
- No browser-grade TLS stack mimicry beyond client-hello fingerprint presets: uTLS presets are in; JS-emulating fingerprints are out.
- No DTLS, SSH, SMB, or VPN (IKEv2) cipher analysis in v1 – v2 candidates.
- No centralised intelligence: pqscan is the point tool in a pipeline, not a database.

### 3.3 Go or Rust: a decision matrix, not a taste

Both languages are viable for a scanner of this shape, and the decision is recorded as a re-scorable matrix with kill-switch criteria rather than a preference [26].

**The Go case.** crypto/tls ships ML-KEM by default: Go 1.24 enables `X25519MLKEM768` by default, 1.25 exposes the negotiated hybrid group in `ConnectionState.CurveID`, and 1.26 adds `SecP256r1MLKEM768` / `SecP384r1MLKEM1024` [18] – the negotiated hybrid group is observable without any C bindings. A TLS 1.0–1.2 client with explicit suite lists makes per-suite "offer-one, observe-accept" probing roughly 20 lines per probe natively. uTLS [24] gives ClientHello-level control – custom `supported_groups` lists, hand-set `key_share` entries, browser presets – and is the probe mechanism for the non-stdlib and obsolete groups (`X25519Kyber768Draft00`, HQC/BIKE experimental groups) that stock stacks refuse to construct. quic-go [23] wraps crypto/tls, so the same group control carries to the UDP plane. The import toolset's ecosystem is Go: the output conventions (streaming JSONL, `-stats`, `-resume`, shard semantics of the zgrab2 shape) carry over as-is, and the cross-compilation and goreleaser release are one command [25]. Go's costs are stated rather than hidden: GC pauses and per-connection memory exceed Rust's at the same in-flight count; the crypto/tls API boundary keeps some 1.2-era suites and all non-stdlib groups invisible to stock stacks (uTLS handles them; SSLv2/SSLv3 are absent from Go entirely – an optional legacy sidecar carries SSLv3, Section 9.2); and one CI constraint must be encoded – setting `Config.CurvePreferences` *replaces* the defaults (a list without 4588 silently kills PQ), and a `go < 1.24` tool line reverts PQ defaults. The CI matrix pins toolchain lines and asserts the engine defaults, so the probe truth stays version-independent.

**The Rust case.** rustls 0.23.22 and later has ML-KEM in core (the `prefer-post-quantum` feature) plus the `rustls-post-quantum` crate (ML-DSA variants, aws-lc-rs provider), and the provider API is the cleanest route in any stack to inject custom KEM groups – a real advantage if requirements later include first-class HQC/BIKE probing. No GC; deterministic tail latency; typically 2–3× smaller steady-state memory per connection; `io_uring`-based networking stacks exist for extreme-scale work; binaries land at 5–8 MB. The costs: TLS 1.0/1.2 legacy-suite enumeration is deliberately restricted (SSLv3-era probing needs vendored OpenSSL bindings – the same sidecar problem with worse ergonomics); the QUIC post-quantum surface (quinn) tracks rustls and is a moving target to pin in CI; tokio adds a second ecosystem to review, and per-probe cancellation in deep mode – thousands of short-lived handshakes – is where async bugs breed; iteration is slower, and the distribution channel (`go install`/Homebrew) matches Go.

**The decision: Go, with uTLS for Hello construction and quic-go for the UDP plane.** The binding constraint for a cipher scanner is socket-parallelism and latency, not per-connection CPU (Section 6.1), so Rust's throughput edge is mostly theoretical here, and Go's ecosystem buys exactly the shape the requirements treat as mandatory. The Rust choice returns only if real measurements show Go missing its throughput SLOs with no fix in reach; the criteria are recorded with the decision (ADR-0001 [26]) – the switch is a decision, not a taste.

## 4. The Tool: Targeting, Modes, and the Probe Set

### 4.1 Targeting and discovery

**Table 2:** The targeting and discovery surface

| Capability | Detail |
|---|---|
| Single-target flags | `-u host`, `-u host:port`, `-u https://host`, `-cidr 192.0.2.0/24`, `ip:port` in one input grammar |
| Lists and stdin | `-l file`, stdin pipe (auto-detected), both mixed; dedupe (exact plus recent-LRU) |
| Ports | `-p 443` default; comma lists; ranges (`8000-8100`); per-port handshake plan |

| Capability | Detail |
|---|---|
| SNI strategies | `auto` (default): SNI when a hostname is present, SNI-less IP probe otherwise; `ptr-then-ip`; `ip-only`; `strict-sni` (non-matching certs fail a readiness rating; the records stay) |
| Reverse DNS | `-ptr` workers resolve PTR per IP (pooled UDP resolver, EWMA timeouts, negative cache) and promote the PTR hostname to an SNI candidate; PTR is enrichment, never a blocker (Section 6.6) |
| Sharding and resume | `-shard i/n` in the zgrab2 form; `-resume` from the WAL; `-stats` live rate/error view |

The grammar-in rule is load-bearing for the chainability contract: hostname, `host:port`, `ip`, `ip:port`, `scheme://host[:port]`, and CIDR notation parse from one reader, and the explicit import adapters (`-from-httpx`, `-from-naabu`, `-from-nuclei`, and masscan stdin auto-parse) accept the reference tools' output verbatim (Section 5.4). Raw pipes work without a flag; the adapters exist where a format needs disambiguation.

## 4.2 The four modes

- **`fast` (default).** One engineered handshake per (host, port): the current browser-grade hello, a uTLS preset with a PQ-hybrid key share preferred [24]. The read: negotiated parameters, certificate chain, ALPN (h2 presence), and the feature observations. An optional one-shot HTTP probe (`-http`) reads status, server header, HSTS, and Alt-Svc – off by default for raw IP sweeps, on for domain workflows.
- **`deep`.** Computes a per-target probe plan and executes it (Section 6.4): version-by-version and suite-by-suite "offer-one / observe-accept" enumeration, group enumeration (native groups plus uTLS-constructed non-stdlib groups), signature-algorithm negotiation, HRR detection, resumption/ticket checks, 0-RTT acceptance, insecure-renegotiation and fallback-SCSV checks. The default deep plan caps probes per target (24) within a time budget; the full matrix is opt-in (`-deep full`).
- **`pq`.** Forced-group probing: send a hello carrying *only* PQ-capable groups – `X25519MLKEM768`, `SecP256r1MLKEM768`, `SecP384r1MLKEM1024`, `MLKEM768`, MLKEM512-era drafts, and the obsolete Kyber drafts as a compatibility oracle – observe which the server accepts, offer the PQ signature schemes (`mldsa44/65/87`, composite ML-DSA), classify certificates, and rate readiness. The offer set's per-probe observations land in the record's deep block in the deep executor's shape (`attempted/ok/rejected` plus findings `probe/result/detail`); `-mode pq -jsonl` carries the offer set exactly as `-mode deep` carries the deep matrix, and the `-text` line rides the same counts tail (`deep=pq:<N>p/<N>ok/<N>rej`, Section 5.5). The mode runs across the whole target set on a schedule – the weekly post-quantum job of Section 5.4.
- **`selftest`.** Runs the tool's own handshake engines against an embedded instrumented endpoint. The read is the local machine's TLS-stack capability matrix – what a fleet *can* reach, not only what servers offer. A long-running `serve` mode (the instrumented endpoint as a service) sits outside the v1 surface.

## 4.3 Protocol coverage

**Table 3:** Protocol coverage in v1

| Plane | Coverage in v1 |
|---|---|
| TLS over TCP | TLS 1.0, 1.1, 1.2, 1.3 full first-class; SSLv3 via the legacy module (opt-in); SSLv2 out of scope until demand (recorded as a tracked gap) |
| HTTP/1.1 over TLS | ALPN `http/1.1` plus the optional single GET probe (`-http`): status, server header, HSTS, Alt-Svc (h3 hint) |
| HTTP/2 over TLS | ALPN `h2` observation; optional negotiated-protocol HTTP probe |

| Plane | Coverage in v1 |
|---|---|
| HTTP/3 / QUIC | TLS 1.3 inside QUIC v1 (RFC 9000) / v2 (RFC 9369) on UDP/443 [28], [29]; PQ group negotiation observed via `ConnectionState` from quic-go [23]; h3 verified by direct QUIC handshake – Alt-Svc is a hint only, the direct probe is truth; the deep census completes one HTTP/3 GET, so "ALPN h3" and "h3 serves requests" read as separate rows |
| QUIC modes | `fast-quic` single handshake; `deep-quic` group enumeration limited to stdlib-supported groups (v1), plus the RFC 9369 transport-version rung (the negotiated version named in the finding) and the functional HTTP/3 rung (the SETTINGS frame, then one GET through the pinned quic-go h3 stack); exotic-group construction inside QUIC deferred to v2 (a forked TLS stack is required) |

### 4.4 The handshake-engine capability checklist

The client plane's capabilities, each stated as what the engine can force and what it thereby observes:

- **Versions.** Force exact min/max; send-only-this-version probes (1.2-era); TLS 1.3 semantics per RFC 9846 [27].
- **Suites.** Offer exact suite lists (single or sets) on TLS 1.0–1.2; the TLS 1.3 suite set is fixed and reported per-acceptance.
- **Groups.** `CurvePreferences` covers the stdlib groups (including 4587/4588/4589); uTLS raw-extension mode drives arbitrary group lists with an explicit `key_share` per candidate [24] – and detects HRR precisely, because a server `hello_retry_request` presence is itself an observable.
- **Signature algorithms.** Force the `signature_algorithms` and `signature_algorithms_cert` lists, including the PQ schemes.
- **Features observed and verified.** ALPN (`http/1.1`, `h2`, `h3`), extended master secret, session tickets, resumption via PSK (and the resumption *rate* over N connections, cryptonice-parity [13]), 0-RTT attempt plus acceptance, fallback SCSV, renegotiation behaviour, compression check.
- **Fingerprints.** JA3/JA4 of the tool's own hello – the transparency half: what the tool sent [31]; JARM of the server (optional flag) [30]; uTLS presets: Chrome, Firefox, Safari, iOS, Gotls.

### 4.5 Certificate analytics

The certificate read is a full chain parse: subject/issuer, SANs, validity, key algorithm and size (RSA, ECDSA curve, Ed25519, Ed448), the signature algorithm of every certificate in the chain, key usage, and SCT/CT presence. **PQC certificate classes come via OIDs:** ML-DSA-44/65/87 (RFC 9881 OIDs `2.16.840.1.101.3.4.3.17–19` [32]), SLH-DSA, composite and hybrid certificate drafts, alongside ECDSA/RSA/other classification. Two cross-observations close the block: the negotiated signature scheme versus the certificate's signature algorithm (mismatches surface), and the issuer hierarchy's PQ readiness – whether the CA is itself PQ or classic, which sets the wait before the leaf's PQ renewal.

### 4.6 Post-quantum classification and the readiness rubric

Classification tables live in generated code (the `internal/classify` regeneration tables), snapshot-sourced from the IANA TLS parameters registry [33]; drift re-generates, and a hand edit never happens. The taxonomy over key exchange: `pq_hybrid` (4585–4590, with the obsolete Kyber drafts labelled `pq_hybrid_draft`), `pq_pure` (the MLKEM512/768/1024 pure groups, draft-era values under draft-ietf-tls-mlkem [34]), `classic` (X25519, P-256/P-384, FFDHE), and `legacy` (SM2 hybrids noted, RSA key exchange, non-compliant groups). The same taxonomy applies over signatures and authentication: `mldsa44/65/87` (0x0904–0x0906), SLH-DSA schemes (draft-reddy-tls-slhdsa [35]), the composite ML-DSA family (draft-reddy-tls-composite-mldsa [36]), the dual-cert `dual_signature_algorithms` extension (draft-yusef-tls-pqt-dual-certs [37]), and classic RSA/ECDSA/EdDSA. The hybrid design framework (draft-ietf-tls-hybrid-design [38]) is the background reference for the hybrid class.

**Table 4:** The readiness rubric (weights fixed in code and printable; score clamped 0–100)

| Signal | Points |
|---|---|
| PQ hybrid (or pure) KEX negotiated in any probed plane | 45 |
| PQ capability observable in offers (PQ groups in the forced probe accepted; PQ sig schemes or dual-cert signalling accepted) | 25 |
| Modern hygiene: TLS 1.3 + forward secrecy + no legacy versions/suites accepted | 15 |
| Certificate/chain PQ signal (PQ or composite cert, PQ-ready issuer) | 15 |

The bands: `pq-ready` at 80 or higher, `pq-ready-hybrid` at 60 or higher, `pq-partial` at 40 or higher, `pq-missing` from 1 to 39. A scan error surfaces as a score of 0 with the `error` field set – a zero-signal scan is an error state, not a grade. The standalone boolean flags ride alongside: `kex_pq`, `auth_pq_signal`, `cert_pq`, `hndl_risk`. The harvest flag deserves its sentence: classic-only key exchange today means the recorded traffic stays decryptable by a future CRQC, and the flag means *harvestable*.

**Table 5:** The PQ codepoint reference (snapshot dated 2026-10-15 from the IANA registry [33]; the build regenerates from the registry CSV – the table is never hand-edited in code)

| Value | Name | Class |
|---|---|---|
| 4585 | SecP256r1MLKEM512 | pq_hybrid |
| 4586 | MLKEM512X25519 | pq_hybrid |
| 4587 | SecP256r1MLKEM768 | pq_hybrid (RFC 10024) |
| 4588 | X25519MLKEM768 | pq_hybrid (RFC 10024, Recommended) |
| 4589 | SecP384r1MLKEM1024 | pq_hybrid (RFC 10024) |
| 4590 | curveSM2MLKEM768 | hybrid (SM2 family) |
| 513 / 514 | MLKEM768 / MLKEM1024 (pure) | pq_pure (draft-ietf-tls-mlkem) |
| 25497 | X25519Kyber768Draft00 | pq_hybrid_draft (OBSOLETE – compat oracle) |
| 25498 | SecP256r1Kyber768Draft00 | pq_hybrid_draft (OBSOLETE – compat oracle) |

The obsolete Kyber draft groups stay probeable deliberately: a server that still accepts 25497 tells the operator which client populations it serves, and the classifier records the acceptance at its own class.

## 5. The Output Contract and Chainability

### 5.1 The dialect rules

The dialect rules are non-negotiable and the whole chainability thesis rests on them: stdin→stdout all the way; one JSON object per line in `-jsonl` / `-json`; the human text default is single-line-per-target in the httpx manner; `-csv` for spreadsheets; exit codes documented; a `--silent` that means *only results*.

### 5.2 The record

The JSONL record is versioned (`schema: 1`); the machine-readable published schema lives at `docs/schema-v1.json`, generated from the goldens and drift-checked in CI [6]. The example is the specification's captured live record (google.com) [6]:

```
{"schema":1,"mode":"fast","target":
{"input":"google.com","host":"google.com","ip":"","port":443,"ptr":"","sni_used":""},"handshake":
{"negotiated":4588,"group_name":"X25519MLKEM768","group_class":"pq_hybrid","suite":"TLS_AES_128_GCM_SHA256","version":"tls-1.3","a
{"attempted":true,"state":"negotiated","negotiated":4588,"group_name":"X25519MLKEM768","group_class":"pq_hybrid","discovered":
{"alt_svc_h3":true,"h3_direct":true}},"cert":{"sig_alg":"ecdsa-with-
SHA256","sig_pq":false,"key_alg":"ECDSA","key_pq":false,"dual":false},"pq":{"score":40,"label":"pq-
partial","hndl":false,"summary":"negotiated_pq_hybrid(+40)"},"error":""}
```


**Table 6:** The record's blocks, in the encoders' contract order (the key order in the JSONL matches; the goldens pin it)

| Block | Fields |
|---|---|
| `target` | `input`, `host`, `ip`, `port`, `ptr`, `sni_used` |
| `handshake` | `negotiated` (group id), `group_name`, `group_class`, `suite`, `version`, `alpn_neg` |
| `quic` | `attempted`, `state` (negotiated · handshaked · refused · mismatch · skipped · udp_blocked), `negotiated`, `group_name`, `group_class`, `discovered{alt_svc_h3, h3_direct}` |
| `cert` | `sig_alg`, `sig_pq`, `key_alg`, `key_pq`, `dual` |
| `pq` | `score` (0–100), `label`, `hndl`, `summary` (the rule trail) |
| `deep` | deep mode and the pq offer set: `mode` (standard · full · pq), `attempted`, `ok`, `rejected`, `findings[]{probe, result (accepted · rejected · refused · error), detail}` – on `mode: "pq"` the findings are the forced PQ offer set's per-probe observations |
| `error` | the per-target error string; an unreachable endpoint is a record, not a process failure, and the `quic` / `cert` / `pq` blocks ride zero-populated on error records – every record class validates against the published schema as-is |

### 5.3 The CSV projection and the output flags

The CSV form is the flat 25-column projection of the same record, and the header order is the contract, pinned by the goldens:

```
target, host, port, ptr, sni, negotiated, group_name, group_class, suite, version, alpn,
quic_state, quic_group, udp_blocked, cert_sig_alg, cert_sig_pq, cert_key_alg, cert_key_pq,
cert_dual, pq_score, pq_label, pq_hndl, pq_summary, error, ts
```

The output flags: `-json`, `-jsonl`, `-csv`, `-text` (default), `-o file`, `-urls` (emit `scheme://host:port` lines for nuclei), `-fields` selection, `-certs-dir` (write end-entity PEMs for later tooling), and `-stream` (unordered streaming, the default) versus `-sorted` (buffered ordering by input order for human review and diffs).

### 5.4 The chaining reference implementations

The composition contract, as concrete commands. All commands run verbatim; pqscan reads upstream output as input lines and never requires a flag unless a format needs disambiguation – the reference recipes in this section are the spec's verbatim implementations [6].

```
subfinder -d example.com -silent | pqscan -mode fast -jsonl -o assets.jsonl
```

```
subfinder -d example.com -silent \
  | httpx -silent -ports 443,8443 -p 443,8443 -no-color \
  | pqscan -mode deep -jsonl -o deep.jsonl
```

(httpx text-mode lines carry host-URI shapes; the built-in parser strips the scheme and the status decorations. `-from-httpx` reads the text dialect only – the contract of the pinned capture command; a httpx `-csv` run refuses here per the documented boundary.)

```
naabu -l domains.txt -p 443,8443,465,993 -silent \
  | pqscan -from-naabu -mode fast -jsonl
```

```
masscan -p 443 -iL cidrs.txt --rate=100000 --open-only -oX - \
  | pqscan -shard 1/4 -mode fast -ptr -rate 20000 -jsonl -o shard1.jsonl
```

```
pqscan -l targets.txt -mode pq -jsonl \
  | jq -r 'select(.pq.label != "pq-ready") | [.target.host, .pq.label, .pq.score, .pq.summary] | @tsv'
```

```
pqscan -l assets.txt -mode fast -urls \
  | nuclei -silent -tags tls
```

The reverse direction – nuclei's findings as pqscan's targets – is the implemented `-from-nuclei` adapter: the pinned capture `nuclei -l hosts.txt -silent -j` writes findings JSONL, each finding's `matched-at` (the `url` field as the fallback) contributes its host:port under the one dedupe, and `pqscan -from-nuclei` reads the lines as targets. The weekly organisation-scale job runs the pq mode's rating over a pinned target list on a schedule – `pqscan -l targets.txt -mode pq -jsonl | diff yesterday.jsonl -` – and pqscan's deterministic ordering (`-sorted`) makes the diffs meaningful.

### 5.5 The CLI surface

The zero-configuration entry is `pqscan -u example.com` – a complete invocation (G6). The flag groups: **targeting** (`-u`, `-l`, `-cidr`, `-p`, `-from-httpx`, `-from-naabu`, `-from-nuclei`, `-shard i/n`, `-resume`); **modes** (`-mode fast|deep|pq|selftest`, `-deep standard|full`, `-quic-always`, `-quic-off`, `-http`); **probes** (`-probe-suite`, `-probe-version`, `-probe-group`, `-probe-sigalgs`, `-client-fingerprint chrome|firefox|safari|ios|gotls`, `-jarm`); **TLS knobs** (`-sni auto|ptr-then-ip|ip-only|strict-sni`, `-ptr`, `-timeout`, `-connect-timeout`, `-retries`); **performance** (`-threads`, `-c`, `-rate`, `-dns-threads`, `-stats`); **output** (`-json`, `-jsonl`, `-csv`, `-urls`, `-text`, `-o`, `-fields`, `-certs-dir`, `-explain`, `-silent`, `-sorted`); shell completions (`-completion bash|zsh`, generated from the same surface the help renders); the opt-in legacy module (`-legacy module=heartbleed,ccs,robot,sslv3`, off by default); and configuration (`-config pqscan.yaml` for org defaults – ports, budgets, probes, output path – with `PQSCAN_*` environment overrides).

The human `-text` line carries the same detail the JSONL carries, as a format-parity surface riding the line when present: `deep=<plan|pq>:<N>p/<N>ok/<N>rej` (the executed plan's counts, and the same tail on `-mode pq`), `cert=<sig>/<key>` with `+pq-sig/+pq-key/+dual`, `ptr=` / `sni=`, and `pq=<label>(<score>)` – with `-explain` printing the plain-language reading under each line. The standing parity census lives in `docs/surface-census.md`: every flag in the specification maps to its usage row and its implemented state, with pinned reference-tool captures, and `scripts/check_spec_usage_parity.sh` guards the census.

## 6. Concurrency Architecture

A cipher scanner is not a web scraper: the work is thousands of short, RTT-dominated, failure-rich network round-trips whose failures carry information (a rejection is a result). The architecture exists to make the *failure case* and the *tail* fast, not only the success path – and the scale is a product requirement, not an implementation note: the tool's identity is "small and fast", so the pipeline design precedes the probes.

## 6.1 Where time goes: the throughput model

Per fast-mode handshake, end to end:

```
L ≈ connect_rtt (1) + tls13_handshake_rtt (1)     [2-RTT floor; TLS 1.2 = 2 handshake RTT]
    + server_compute (~50–300 ms typical, CDN edge; +1 RTT when HRR is required)
    + client_crypto (MLKEM-768 encapsulation ~0.05–0.15 ms; negligible)
```

At an RTT of 60 ms, L lands between 170 and 400 ms. The throughput of a pool of W concurrent connections is `T ≈ W / L` – not cores times something. Concretely: W = 50,000 in-flight with L = 0.3 s gives a ceiling near 165,000 handshakes/s. The real-world caps bind first: DNS, dial syscalls, client crypto and GC, and etiquette. Client-side crypto measures at 0.2 to 0.6 ms per handshake in Go, so the CPU-bound ceiling is roughly 2,000 to 4,000 handshakes/s per core. The conclusion drives every budget in Section 6.3: **a bounded-parallel pipeline, with W tuned to the CPU and the ethics caps – and the language choice is then not the constraint.** The model is re-measured, never assumed.

## 6.2 The pipeline shape: five bounded stages

The pipeline is five stages, all bounded, all streaming: **ingest** (grammar parse plus dedupe), **enrich** (shard filter plus the DNS pool doing A/AAAA/PTR), **scheduler** (a fair target queue plus plan builder), the two **engines** (TCP: crypto/tls + uTLS; QUIC: quic-go), and **classify/score/WAL/encoders** streaming jsonl, csv, text, and urls out. The design rules:

- **No stage ever blocks its producer on a slow individual item.** Each stage is a goroutine pool pulling from a bounded channel. Ingest never waits on DNS; the scheduler never waits on a handshake; the encoder never waits on a handshake (the WAL takes the writes, then the encoders drain).
- **No unordered goroutine spread.** Concurrency is worker pools over channels with explicit budgets – never one goroutine per target with unlimited fan-out.
- **Streaming is the default.** A record emits the moment its target's plan completes; `-sorted` is the only buffered mode.
- **Cancellation is pervasive.** Every probe carries the deadline ladder; every stage honours context cancellation; Ctrl-C drains, the WAL stays consistent, exit code 130.

## 6.3 Budgets and fairness

**Table 7:** The budgets (the numbers that make it fast *and* safe)

| Budget | Default | Rationale |
|---|---|---|
| Global in-flight connections | 50,000 (CPU-tuned by startup probe on the box) | `T ≈ W/L` ; 50k sits at roughly 2 GB RAM with 20–40 KB per connection |
| TCP in-flight | 40,000 | QUIC needs more RAM per connection |
| QUIC in-flight | 10,000 | per-connection goroutines plus the timer wheel are heavier |
| Deep-mode probes per target (concurrent) | 6–8 | one host is not 64 handshakes at once; the low depth keeps server-side rate limiting quiet |
| Per-subnet (/24) concurrent | 64 | NAT and policy devices fail closed; breakers catch the rest |
| Global rate cap | 20,000 handshakes/s ( `-rate` for more) | the etiquette default; internet recipes use 5 connections or fewer per IP per second |

| Budget | Default | Rationale |
|---|---|---|
| DNS in-flight | 256 (pooled resolver with EWMA) | PTR storms must not starve handshakes – a separate budget |
| Handshake deadline ladder | connect 2 s / handshake 4 s / probe-total 8 s (with ±20 % jitter) | jitter de-synchronises retries; the ladder bounds the tail |
| Retries | 1, transient-only (RST/timeout), never for protocol rejections | rejections are results, not errors |
| Result channel | 10,000 entries before backpressure | the WAL keeps flushes off the hot path |

**Adaptive concurrency (AIMD) plus circuit breakers.** The global W ramps from 1,000 toward its cap; on an error-rate spike (the timeout/conn-fail ratio) it halves, and recovers additively on success streaks. Per-/24 and per-ASN breakers: K consecutive timeouts and the bucket cools for minutes; a cooled bucket's results carry `skipped` with a reason – never silent. The target queue is weighted-fair, so one pathological /24 cannot starve the run.

**Fairness inside a target (deep mode).** The per-target plans run through a local scheduler: the probes are cheap stateless handshakes, so the depth is bounded by plan length × per-probe latency – never by a lock or a queue. The probe results feed the plan builder (Section 6.4), which prunes redundant probes in flight.

## 6.4 Deep-mode scheduling: the probe plan

Deep mode is a per-target directed plan, not a fixed sequence. Phase 1 is the profile handshake; its result prunes phase 2 (a suite already negotiating is skipped; groups the server re-routed via HRR are marked). If the profile negotiated a `pq_hybrid` group, the pq-forced probes are skipped and the plan moves to the remaining groups and signature algorithms; otherwise the PQ forced-group offer set runs (4588, 4587, 4589, the MLKEM pure groups, the drafts). The suite matrix follows – 1.0–1.2 offer-one/observe-accept, and the 1.3 fixed set reported per-acceptance – then the signature-algorithm matrix including `mldsa` and composites, then the feature probes (HRR, 0-RTT, resumption over N connections, SCSV, renegotiation), then the QUIC plane if hinted or `-quic-always`, then classify/score/emit. The default plan caps at 24 probes per target; `-deep full` unlocks roughly 120. One rule carries the whole design: **the PQ offer set runs even on `classic` servers** – whether a stack accepts 4588 when forced is the actionable fact across every target, not a fact reserved for servers that advertise it.

## 6.5 The QUIC engine

The QUIC plane runs quic-go [23] with a `tls.Config` carrying the same group and signature control as TCP; versions v1/v2, ALPN `h3`. The deep census plans two rungs on every QUIC-gated target: the RFC 9369 transport-version rung (the v2-only offer on UDP/443, with the negotiated version named in the finding) and the functional HTTP/3 rung – the SETTINGS frame, then one HTTP/3 GET through the pinned quic-go h3 stack. A negotiated handshake that carries no functional h3 layer records its own finding class. QUIC probes switch on in two cases: the explicit `-quic-always`, or an Alt-Svc h3 hint from fast mode (the hint gate); raw-internet mode runs QUIC only for hinted hosts by default (UDP half-open scans are a v2 idea, not the default). The in-flight budget is separate (10,000) because quic-go's per-connection goroutine and timer cost sits near 5× TCP's, and the budget is auto-derived at startup from the measured constants, like TCP's. Networks that block UDP outbound are detected in probes and reported as `udp_blocked` rather than retried.

### 6.6 No head-of-line anywhere

The discipline list, stated as invariants:

- **DNS/PTR enrichment never delays the handshake stream.** SNI-less IP handshakes run in parallel; the PTR result upgrades the record when it arrives in time – `-sni ptr-then-ip` re-handshakes only when the PTR arrives after the initial SNI-less attempt reported a default-cert mismatch risk.
- **The encoders never wait on disk.** The WAL is append-only and fsync-batched (every 1,000 records or every 1 s).
- **The scheduler never waits on a slow target.** The plans finish through the deadline ladder.
- **The tool never waits on the last slow target.** The Ctrl-C drain takes seconds, within the ladder.
- **`-resume` replays the WAL.** The completed-target set is a compact on-disk set (exact to 4 M entries, then spilled), so an internet pass interrupted at hour four resumes in seconds.

### 6.7 Memory, allocation, and GC discipline

Handshake buffers ride `sync.Pool` (4–16 KB), hello templates pre-sized per fingerprint preset, and codepoint→name maps are static arrays – no `Sprintf` in the hot path. The JSONL encoder is hand-rolled for the hot record (no reflection), giving roughly 3× less allocation than the `encoding/json` marshal paths. `GOMEMLIMIT` is set by default from the W budget; `GOGC` 300; optional `GOGC=off` plus `FreeOSMemory` exists for long internet passes – benchmarked before release, never assumed. The standing target is under 4 GB RSS at 50,000 TCP plus 10,000 QUIC in-flight including the result and WAL buffers, measured in the CI performance suite (Section 7.4).

### 6.8 Benchmarks and SLOs

**Table 8:** The SLO table (CI-enforced per release cut, measured, never assumed)

| Metric | SLO |
|---|---|
| Fast-mode handshakes/s/core against cloudflare.com | ≥ 2,500 |
| Fast-mode aggregate, 16 cores, rate 20k/s | sustained 30 min, W stable |
| Deep-mode probes/s (mixed real hosts) | ≥ 100 |
| Deep full-matrix per host, 60 ms RTT | ≤ 25 s p50 |
| QUIC handshakes/s/core | ≥ 1,000 |
| p95 per-target completion, fast mode under load | < 15 s |
| RSS at 50k/10k in-flight | < 4 GB |
| Resume replay of a 10 M-record WAL | < 60 s |

**Comparative baselines are recorded, not claimed:** zgrab2, sslyze, cryptonice, and gnutls-cli-debug on per-target depth, and httpx on raw churn; the repository publishes the harness and the numbers [6]. zgrab2 leads on raw churn; this tool leads on per-target depth at chainable scale – the position Section 2.3 holds, with measurements attached.

# 7. Test and Verification Strategy

## 7.1 Test-first is the merge gate

Every behavioural change lands as a watched RED→GREEN pair (ADR-0002 [39]): the failing test exists and runs before the implementation, the RED output is recorded verbatim in the pull request, and CI re-proves the green suite. Rejections and observed hostile input are results, and their tests are fixtures first.

## 7.2 Correctness fixtures

Three fixture families bound correctness. **The badssl.com subdomain matrix** [40] – expired, untrusted, suite-hostile, version-hostile, mismatch, RC4/3DES hosts – gives expected results checked per probe, and regressions fail CI. **The PQ ground truth** runs against Google-hosted and Cloudflare-hosted PQ test endpoints (known 4588-negotiating), a local OpenSSL/BoringSSL TLS 1.3 configuration matrix held as owned fixtures (suite/group/sigalg per configuration), and local Go servers serving ML-DSA/composite certificates where the registries permit. **The differential check** runs gnutls-cli-debug and sslyze [7], [20] against pqscan on the fixture matrix: the ratings must agree, or a written exception records why the pqscan read is the correct one – for instance the PQ groups gnutls cannot construct.

## 7.3 Protocol truth

QUIC results are validated with Wireshark-visible handshakes on fixtures [41]; h3 presence is cross-checked against Alt-Svc, and a mismatch is reported explicitly – "Alt-Svc says h3, direct QUIC says no" is a finding, not an error (the direct probe is truth, Section 4.3). TLS-version and suite decisions are double-checked against raw hello parsing when uTLS and crypto/tls disagree [24].

## 7.4 Performance and resilience

The CI performance suite enforces the SLO table (Section 6.8) against a local multi-port TLS server farm plus a controlled-latency container (`tc`), with no external network dependence. Chaos probes cover mid-handshake RST floods, black-holing, resolver failures, and WAL unmount; the invariants are no crash, WAL consistency, resume replay, and breakers that trip and recover.

## 7.5 Rate safety by construction

Conservative defaults ship: the 20k/s global rate cap, per-subnet concurrency, the deadline ladder with jitter, and breaker cool-downs (Table 7); the `-rate` flag raises the pace inside controlled environments. The README carries the legal and ethical notes for mass scanning, the opt-out contact, and framing in the zmap/masscan practice [11], [12]. The scan operator owns the ethics; the tool enforces the conservative defaults by construction, never by a policy claim.

# 8. Success Metrics

- **Adoption.** The tool runs as the TLS/PQC stage in real chains; star counts are not the measure – the watch is external issues and recipe pull requests.
- **Correctness.** A fixture-matrix pass rate of 100 %; differential exceptions documented and under 5 % of categories.
- **Performance.** The SLO table green in CI per cut; the real-world read: one box, fast mode, 10 M targets in under 24 hours at the default caps.

- **Utility.** The scheduled PQ job (mode `pq`) gives diff-able ratings across the target set; adoption starts with the maintainer's own target set.

## 9. Risks, Limitations, and Future Work

### 9.1 The risk register

**Table 9:** The risk register with mitigations

| Risk | Mitigation |
|---|---|
| crypto/tls/uTLS drift or API changes break group forcing | the pinned capability run re-proves both stacks; CI runs the capability matrix nightly; vendor-lock fences in go.mod; the IANA regen pipeline makes codepoint drift a rebuild |
| SNI-less IP handshakes return CDN default certs – misleading ratings | the `-strict-sni` rating flag; SNI-less records carry `sni_used=false` and take the `provisional` badge unless strict mode |
| Middleboxes choke on large PQ ClientHello (the tldr.fail class [42]) | recorded as the `clienthello_too_large` observed state; fast mode can fall back to a classic hello and re-probe (the `-pq-fallback` flag, default on fast mode for raw IP scans) |
| quic-go budget blowout on internet scale | a separate QUIC budget, hint-gated probing, live measurements driving W_q |
| Deep mode on millions of hosts is nonsense time | explicit guidance plus the fast-mode default; the sampling recipe (deep on top-N ASNs, fast everywhere); rate and etiquette docs |
| Legal or abusive use of mass scanning | the Section 7.5 defaults and docs; no bundled "internet scan" one-liner; shard support, but the user owns the ethics |
| Single-maintainer risk | MIT licence, small module surface, the probe matrix as data (not code), so contributors add probes without touching engines |

### 9.2 Stated limitations

The limitations are stated as design facts. **SSLv3 reaches the tool through the opt-in legacy module** (Section 4.3), and SSLv2 stays out of scope until demand exists (recorded as a tracked gap, not a feature). **No DTLS, SSH, SMB, or IKEv2 analysis ships in v1** – these are the named v2 candidates. **Inside QUIC, deep-mode group enumeration is limited to stdlib-supported groups**, and exotic-group construction inside QUIC is deferred to v2 because a forked TLS stack is required; UDP half-open scans stay a v2 idea rather than the default. **The `serve` mode** (the instrumented endpoint as a long-running service) sits outside the v1 surface. **The PQ pure groups (513/514) remain draft-era codepoints** and HQC carries no TLS codepoint yet [4], [34] – the classifier and the regen tables, not the scanner logic, absorb the registry movement. **Fingerprint mimicry stops at the uTLS presets** – JS-emulating fingerprints are out of scope by design. And there is no centralised intelligence: pqscan is the point tool in a pipeline, not a database.

### 9.3 Future work

The v2 lanes follow directly from the boundary: DTLS, SSH, SMB, and IKEv2 cipher analysis; exotic-group QUIC construction once a forked TLS stack is worth owning; the UDP half-open discovery mode; the long-running `serve` mode; and the continued PQ registry movement (composite certificate types, the dual-cert extension, and the HQC codepoint when one arrives) landing through the regeneration pipeline rather than through hand edits [33], [36], [37].

## 10. Ethics, Authorship, and Use

This is a tool for reading configuration, and it is built for use against systems the operator is entitled to read. The specification carries the boundary statements, and this paper mirrors their sense: scanning targets are engagements with written authorisation, academic research, and defender validation of one's own estate; the rate defaults, the deadline ladder, the breakers, and the documented multi-box sharding recipe are the tool's own ethics-of-construction (Section 7.5), and the operator owns the rest – including the law of their jurisdiction and the opt-out contact carried in the README.

*Authorship* – Shaineel Singh is the sole author and maintainer (shain.singh@owasp.org).

*Authorised use* – the tool serves authorised security work only: engagements with written authorisation, academic research, and defender validation.

*Funding:* none.

*Conflicts of interest:* none declared.

## 11. Conclusion

pqscan is an argument about one altitude: the cipher-configuration plane of TLS and QUIC, read exhaustively, classified against the post-quantum registries, rated on a fixed rubric, and delivered as a pipe. The argument's claims: that the NIST wave turned cipher configuration into the layer where post-quantum readiness actually lives, and that no existing tool shape – exhaustive, fast, or accessible – covers it at chainable scale; that the gnutls-cli-debug probe class, rethought as a scripted, parallelised plan bounded by explicit budgets, can carry that altitude at discovery-tool speeds; that classification is a data discipline – tables generated from the IANA registry, taxonomy fixed, rubric printable – rather than scanner cleverness; that chainability is a contract (schema-versioned records, column-pinned CSV, verbatim pipe recipes) and composition beats rebuilding the discovery layer the ecosystem already owns; and that the concurrency architecture – a bounded five-stage pipeline on a measured `T ≈ W/L` model, with adaptive concurrency, breakers, and a no-head-of-line discipline – is what makes a 50,000-connection scanner safe rather than merely fast. The claims are carried by the verification strategy: test-first merges, fixture matrices, differential evaluation, chaos probes, and an SLO table re-proven in CI each cut. The tool serves authorised security work; the ethics sit in the defaults by construction and with the operator by decision.

The work ahead is the same work the tool was designed to make cheap: watching a fleet's readiness ratings move week by week as the registries, the CAs, and the server estates move – with the diffs in hand, from a tool that fits in a pipe.

## 12. References

[1] NIST. “FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM).” nist.gov, 2024.

[2] NIST. “FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA).” nist.gov, 2024.

[3] NIST. “FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA).” nist.gov, 2024.

[4] NIST. “HQC – the additional key-encapsulation mechanism selected in 2025.” nist.gov. (Watchlist in the pqscan taxonomy: no TLS codepoint yet.)

[5] RFC 10024. “Registries for the ECDHE-MLKEM hybrid key-exchange groups.” rfc-editor.org. (Groups 4587, 4588 – Recommended – and 4589.)

[6] Singh, S. “pqscan – Product Requirements Document” (v0.3, 2026-10-15). docs/spec.md, shsingh/pqscan. (The companion specification; this paper’s single source. Captured-record example, published schema docs/schema-v1.json, and the parity census docs/surface-census.md.)

[7] GNU TLS. “gnutls-cli-debug: the GnuTLS debug client.” gnutls.org. (The reference-standard exhaustive single-server probe set.)

[8] drwetter. “testssl.sh.” github.com/drwetter/testssl.sh. (Shell-based exhaustive checks; portable.)

[9] Ruhr-Universität Bochum / Trust Centre. “TLS-Scanner: Research-grade exhaustive protocol analysis.” github.com/RUB-NDS/TLS-Scanner.

[10] zmap. “zgrab2: the modular application-layer scanner.” github.com/zmap/zgrab2. (Streaming output posture; shard/resume/stats semantics borrowed.)

[11] Graham, R. “masscan: the internet-scale SYN port scanner.” github.com/robertdavidgraham/masscan.

[12] zmap. “zmap: the fast internet-wide scanner.” github.com/zmap/zmap.

[13] F5 Labs. “cryptonice: the flexible TLS/HTTP/DNS/JARM scanner.” github.com/f5devcentral/cryptonice. (Readable report shape; resumption-rate check parity.)

[14] rbsec. “sslscan.” github.com/rbsec/sslscan. (The accessible single-target scanner posture.)

[15] projectdiscovery. “subfinder: fast passive subdomain enumeration.” github.com/projectdiscovery/subfinder.

[16] projectdiscovery. “httpx: the HTTP toolkit with pipe-native output.” github.com/projectdiscovery/httpx.

[17] projectdiscovery. “nuclei: template-driven vulnerability scanning.” github.com/projectdiscovery/nuclei.

[18] Go project. “crypto/tls release notes: ML-KEM defaults (Go 1.24 enables X25519MLKEM768 by default; 1.25 exposes ConnectionState.CurveID; 1.26 adds SecP256r1MLKEM768 / SecP384r1MLKEM1024).” go.dev/doc.

[19] projectdiscovery. “naabu: the port scanner.” github.com/projectdiscovery/naabu.

[20] nabla-c0d3. “sslyze: the SSL/TLS analysis library.” github.com/nabla-c0d3/sslyze.

[21] Censys. search.censys.io. (Mass passive TLS datasets; PQ group observations at dataset scale.)

[22] Shodan. shodan.io. (Remote-only passive visibility; the complementary vantage to active probing.)

[23] quic-go. “quic-go: a QUIC implementation for Go.” github.com/quic-go/quic-go. (The UDP-plane engine; TLS-over-QUIC group control via tls.Config.)

[24] refraction-networking. “uTLS: ClientHello-level TLS fingerprint control.” github.com/refraction-networking/utls. (Raw-extension group lists, explicit key_share, browser presets.)

[25] goreleaser. goreleaser.com. (One-command cross-compilation and release packaging.)

[26] pqscan repository documentation. “ADR-0001: the language decision (Go, with uTLS and quic-go) and the recorded kill-switch criteria.” docs/adr/.

[27] RFC 9846. “TLS 1.3.” rfc-editor.org. (The TLS 1.3 protocol reference in the spec’s registry statements.)

[28] RFC 9000. “QUIC: A UDP-Based Multiplexed and Secure Transport (QUIC v1).” rfc-editor.org.

[29] RFC 9369. “QUIC Version 2.” rfc-editor.org. (The transport-version rung target.)

[30] Salesforce. “JARM: active TLS server fingerprinting.” github.com/salesforce/jarm.

[31] FoxIO. “JA4 (and the JA3 lineage): TLS client fingerprint specifications.” github.com/FoxIO-LLC/ja4. (Applied to the tool’s own hello: what the tool sent.)

[32] RFC 9881. “ML-DSA certificate OIDs (2.16.840.1.101.3.4.3.17–19).” rfc-editor.org.

[33] IANA. “TLS Parameters registry (supported groups, signature schemes).” iana.org/assignments/tls-parameters. (The regeneration-snapshot source and the codepoint drift surface.)

[34] IETF. “draft-ietf-tls-mlkem: pure ML-KEM key exchange groups.” datatracker.ietf.org. (Values 513/514, draft-era.)

[35] IETF. “draft-reddy-tls-slhdsa: SLH-DSA signature schemes for TLS.” datatracker.ietf.org.

[36] IETF. “draft-reddy-tls-composite-mldsa: the composite ML-DSA family for TLS.” datatracker.ietf.org.

[37] IETF. “draft-yusef-tls-pqt-dual-certs: the dual_signature_algorithms extension.” datatracker.ietf.org.

[38] IETF. “draft-ietf-tls-hybrid-design: the hybrid key-exchange design framework.” datatracker.ietf.org.

[39] pqscan repository documentation. “ADR-0002: the watched RED→GREEN merge gate.” docs/adr/. (Failing test first; RED output verbatim in the PR; CI re-proves green.)

[40] badssl.com. (The deliberate-failure TLS fixture host; the subdomain matrix source.)

[41] Wireshark. wireshark.org. (QUIC handshake validation on fixtures.)

[42] tldr.fail. (The oversized-ClientHello middlebox failure class.)

Reading map. The post-quantum substrate is [1]–[5], [27]–[29], [31], [32], [34]–[38]; the surveyed field and the composition posture [7]–[24] with the passive datasets [21], [22]; the language and release posture [18], [23]–[26]; the rubric and codepoint discipline [33]; and the verification fixtures [40]–[42]. The companion specification [6] is the single canonical source for every statement in this paper.